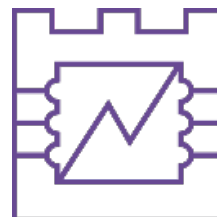




POLITECHNIKA KRAKOWSKA
im. T. Kościuszki

Wydział Inżynierii Elektrycznej i Komputerowej

Katedra



Kierunek studiów:

Specjalność:

STUDIA STACJONARNE

PRACA DYPLOMOWA

INŻYNIERSKA / MAGISTERSKA

Imię NAZWISKO

TEMAT PRACY (w języku polskim)

TEMAT PRACY (w języku angielskim)

Opiekun pracy:

Imię i nazwisko opiekuna pracy

Kraków, 20.....

Spis treści

Streszczenie	4
Wykaz słów kluczowych	5
Wykaz ważniejszych oznaczeń i skrótów	6
1. Wstęp	7
1.1. Wprowadzenie	7
1.2. Cel i zakres pracy	7
1.3. Przegląd zawartości pracy	7
2. Podstawy teoretyczne	8
2.1. System Home Assistant	8
2.1.1. Filozofia i Architektura	8
2.1.2. Kluczowe Pojęcia i Model Danych	8
2.1.3. Silnik Automatyzacji	9
2.1.4. Ekosystem i Rozszerzalność	9
2.1.5. Interfejs Użytkownika i Komunikacja	9
2.2. Rozwiązania sprzętowe dla Home Assistant	10
2.2.1. Kontekst historyczny: Fundamenty informatyki	11
2.2.2. Komputery jednopłytkowe (SBC)	11
2.2.2.1. Raspberry Pi	11
2.2.2.2. ODROID-C4	12
2.2.3. Dedykowany sprzęt: Home Assistant Green	12
2.2.4. Rozwiązania wysokowydajne i klasy enterprise	13
2.2.4.1. Komputery osobiste x86-64	13
2.2.4.2. Nowoczesne komputery Mini PC: Apple Mac Mini	13
2.3. Protokół MQTT	14
2.3.1. Architektura i zasada działania	14
2.3.2. Mechanizmy zapewnienia jakości usług (QoS)	15
2.3.3. Kluczowe funkcje protokołu	15
2.3.4. MQTT w systemie Home Assistant	16
2.3.5. Bezpieczeństwo w MQTT	16
2.4. Docker	16
2.4.1. Konteneryzacja a Wirtualizacja	16
2.4.2. Architektura Systemu Docker	17
2.4.3. Kluczowe Pojęcia i Obiekty	17
2.4.4. Zalety Wykorzystania Dockera w Projektach Inżynierskich	18
3. Praca własna	19
3.1. Przygotowanie środowiska serwerowego	19
3.1.1. Instalacja systemu operacyjnego Debian na laptopie Lenovo T470	19
3.1.1.1. Charakterystyka sprzętowa	19
3.1.1.2. Przygotowanie nośnika instalacyjnego	19
3.1.1.3. Konfiguracja BIOS/UEFI	19
3.1.1.4. Przebieg instalacji	20

3.1.1.5. Konfiguracja poinstalacyjna	20
3.1.2. Konteneryzacja środowiska przy użyciu platformy Docker	20
3.1.2.1. Instalacja silnika Docker Engine	20
3.1.2.2. Konfiguracja poinstalacyjna	21
3.1.2.3. Weryfikacja poprawności instalacji	21
3.1.2.4. Wykorzystanie Docker Compose	21
3.2. Uruchomienie brokera MQTT	22
3.3. Postawienie Home Assistant	22
3.4. Analiza ruchu sieciowego	22
4. Podsumowanie	23
Bibliografia	24

Streszczenie

Wykaz słów kluczowych

Wykaz ważniejszych oznaczeń i skrótów

1. Wstęp

1.1. Wprowadzenie

W dzisiejszych czasach, wraz z dynamicznym rozwojem technologii i globalizacji, wiele dziedzin życia ulega znaczącym zmianom. Wprowadzenie nowych technologii, zmiany społeczne oraz rosnące wymagania stawiane przed jednostkami i organizacjami sprawiają, że konieczne jest ciągle dostosowywanie się do nowych warunków. W tym kontekście, niniejsza praca dyplomowa ma na celu zbadanie i analizę wybranych aspektów związanych z tymi zmianami, a także przedstawienie propozycji rozwiązań i rekomendacji, które mogą przyczynić się do lepszego zrozumienia i efektywnego radzenia sobie z wyzwaniami współczesnego świata.

1.2. Cel i zakres pracy

Celem niniejszej pracy dyplomowej jest...

1.3. Przegląd zawartości pracy

...

2. Podstawy teoretyczne

2.1. System Home Assistant

Home Assistant to zaawansowana platforma typu open-source, służąca do scentralizowanego zarządzania automatyką domową. System ten pełni rolę lokalnego kontrolera (tzw. bramki lub huba), integrującego różnorodne urządzenia i usługi w jedną, spójną strukturę. Kluczowym wyróżnikiem Home Assistant jest jego niezależność od dostawców chmurowych oraz nacisk na prywatność użytkownika i lokalne przetwarzanie danych.

Projekt został zapoczątkowany przez Paulusa Schoutsena we wrześniu 2013 roku jako aplikacja napisana w języku Python. Od tamtej pory, dzięki aktywnemu wsparciu społeczności zgromadzonej wokół serwisu GitHub, Home Assistant stał się jednym z najbardziej rozbudowanych i najczęściej aktualizowanych systemów automatyki domowej na świecie.

2.1.1. Filozofia i Architektura

Fundamentem Home Assistant jest idea „Local Control” (sterowanie lokalne) oraz „Privacy First” (prywatność przede wszystkim). W przeciwieństwie do wielu komercyjnych rozwiązań (np. Google Home, Amazon Alexa), Home Assistant dąży do tego, aby cała logika sterowania oraz przechowywanie danych odbywało się bezpośrednio w sieci lokalnej użytkownika. Takie podejście nie tylko zwiększa bezpieczeństwo, ale również zapewnia ciągłość działania systemu w przypadku awarii połączenia internetowego.

Architektura systemu opiera się na zdarzeniach (event-driven architecture). Wszystkie interakcje, zmiany stanów urządzeń czy wyzwacze automatyzacji są procesowane przez centralną magistralę zdarzeń (Event Bus).

2.1.2. Kluczowe Pojęcia i Model Danych

W celu zachowania interoperacyjności między tysiącami różnych urządzeń, Home Assistant wykorzystuje sformalizowany model danych, którego podstawą są następujące komponenty:

- **Integracje (Integrations):** Są to moduły oprogramowania umożliwiające komunikację Home Assistant z zewnętrznymi platformami, urządzeniami lub usługami API. Na przykład integracja „Philips Hue” pozwala systemowi porozumiewać się z mostkiem Hue Bridge, a tym samym sterować oświetleniem tego producenta.
- **Urządzenia (Devices):** Reprezentują fizyczne lub logiczne obiekty w systemie. Jedno urządzenie (np. wielofunkcyjny czujnik) może składać się z wielu encji (np. czujnika temperatury, czujnika wilgotności i czujnika ruchu).
- **Encje (Entities):** To najmniejsze jednostki danych. Każda encja reprezentuje konkretną funkcję lub właściwość (np. stan włącznika, odczyt temperatury). Encje posiadają stany (States) oraz atrybuty, które są przechowywane w bazie danych systemu.
- **Obszary (Areas):** Pozwalają na logiczne grupowanie urządzeń i encji według ich fizycznej lokalizacji w domu (np. salon, kuchnia, sypialnia). Umożliwia to łatwiejsze

zarządzanie grupami urządzeń, np. wyłączenie wszystkich światel v danym pomieszczeniu jedną komendą.

2.1.3. Silnik Automatyzacji

Home Assistant oferuje potężny mechanizm automatyzacji, który pozwala na tworzenie złożonych scenariuszy interakcji między urządzeniami. Każda automatyzacja składa się z trzech podstawowych elementów:

1. **Wyzwalacze (Triggers):** Zdarzenia, które inicjują sprawdzenie automatyzacji (np. wykrycie ruchu, konkretna godzina, zachód słońca).
2. **Warunki (Conditions):** Opcjonalne kryteria, które muszą zostać spełnione, aby działanie zostało wykonane (np. „tylko jeśli jest po godzinie 22:00” lub „tylko jeśli nikogo nie ma w domu”).
3. **Działania (Actions):** Operacje wykonywane po spełnieniu warunków (np. włączenie światła, wysłanie powiadomienia na telefon).

Uzupełnieniem automatyzacji są **Skrypty**, będące sekwencjami działań wywoływanych na żądanie, oraz **Sceny**, które pozwalają na natychmiastowe przywrócenie określonego stanu wielu urządzeń jednocześnie (np. scena „Kino” przyciemniająca światła i zasłaniająca rolety).

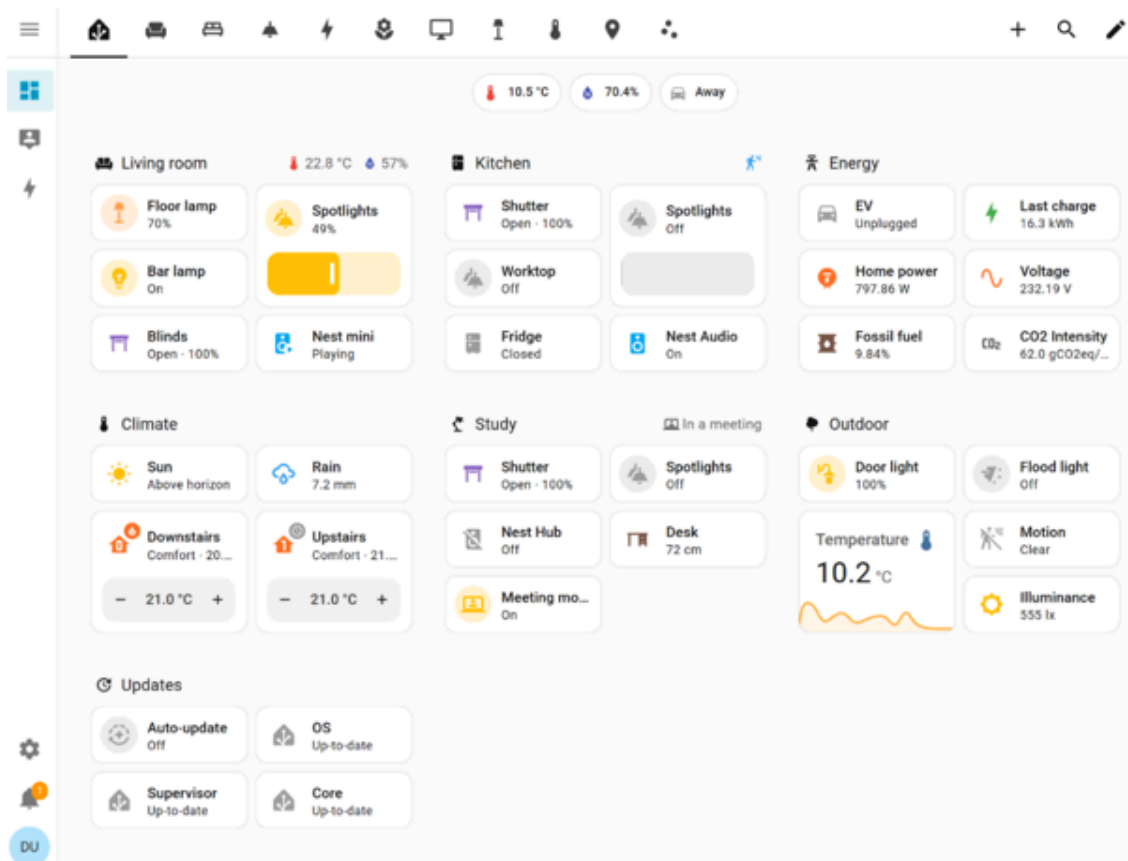
2.1.4. Ekosystem i Rozszerzalność

System Home Assistant wyróżnia się ogromnym ekosystemem, który znacznie wykracza poza standardowe integracje:

- **Dodatki (Add-ons):** To kontenery Dockerowe uruchamiane równolegle z Home Assistant, dostarczające dodatkowe usługi, takie jak serwer MQTT (Mosquitto), bazy danych (InfluxDB) czy narzędzia do wizualizacji (Grafana).
- **HACS (Home Assistant Community Store):** Nieoficjalne repozytorium społecznościowe, pozwalające na instalację niestandardowych integracji oraz elementów interfejsu graficznego, które nie zostały jeszcze włączone do oficjalnej dystrybucji.
- **Blueprints:** Gotowe szablony automatyzacji, które użytkownicy mogą łatwo importować i konfigurować bez konieczności samodzielnego tworzenia logiki od podstaw.

2.1.5. Interfejs Użytkownika i Komunikacja

Głównym sposobem interakcji z systemem jest pulpit nawigacyjny (Dashboard), oparty na silniku Lovelace. Jest on w pełni konfigurowalny i responsywny, co pozwala na tworzenie przejrzystych interfejsów sterowania dostępnych zarówno przez przeglądarkę WWW, jak i dedykowane aplikacje mobilne na systemy Android i iOS. Przykładowy wygląd pulpitu nawigacyjnego przedstawiono na rysunku Rysunek 1.



Rysunek 1: Przykładowy pulpit nawigacyjny systemu Home Assistant

Home Assistant wspiera szeroki wachlarz protokołów komunikacyjnych, co czyni go prawdziwym centrum inteligentnego domu. Do najpopularniejszych należą:

- **Wi-Fi:** Wykorzystywany przez wiele urządzeń IoT, często poprzez protokół MQTT.
- **Zigbee i Z-Wave:** Energooszczędne protokoły typu mesh, idealne dla czujników bateryjnych.
- **Matter:** Najnowszy standard interoperacyjności, wspierany przez największych graczy rynkowych, którego Home Assistant był jednym z wczesnych propagatorów.

Dzięki swojej elastyczności i otwartości, Home Assistant jest obecnie uznawany za jedną z najbardziej wszechstronnych platform do budowy zaawansowanych systemów Smart Home, stanowiąc doskonałą bazę do badań naukowych i realizacji projektów inżynierskich.

2.2. Rozwiązania sprzętowe dla Home Assistant

Wybór odpowiedniej platformy sprzętowej jest fundamentalną decyzją przy wdrażaniu systemu automatyki domowej opartego na Home Assistant. Jako platforma open-source, Home Assistant został zaprojektowany tak, aby był niezależny od sprzętu, wspierając szeroką gamę architektur — od energooszczędnych komputerów jednopłytkowych opartych na architekturze ARM po wysokowydajne serwery x86-64. Wybór zależy od kilku czynników: złożoności logiki automatyzacji, liczby zintegrowanych urządzeń, potrzeby lokalnego przetwarzania danych (takich jak rozpoznawanie obrazu) oraz pożądanego poziomu niezawodności i redundancji systemu.

2.2.1. Kontekst historyczny: Fundamenty informatyki

Aby zrozumieć obecny krajobraz sprzętu do automatyki domowej, warto spojrzeć wstecz na początki komputerów osobistych. Procesor Intel 8086, wprowadzony w 1978 roku, ustanowił architekturę x86, która do dziś pozostaje dominującą siłą w informatyce. Choć sam 8086 jest zbyt prymitywny dla nowoczesnej automatyki domowej, jego dziedzictwo żyje w potężnych serwerach x86-64, których wielu entuzjastów używa do uruchamiania złożonych instancji Home Assistant.



Rysunek 2: Procesor Intel 8086, który zapoczątkował architekturę wciąż używaną w nowoczesnych, wysokiej klasy serwerach domowych.

2.2.2. Komputery jednopłytkowe (SBC)

Komputery jednopłytkowe stały się najpopularniejszym wyborem dla użytkowników Home Assistant ze względu na niski pobór mocy, kompaktowe rozmiary i cichą pracę.

2.2.2.1. Raspberry Pi

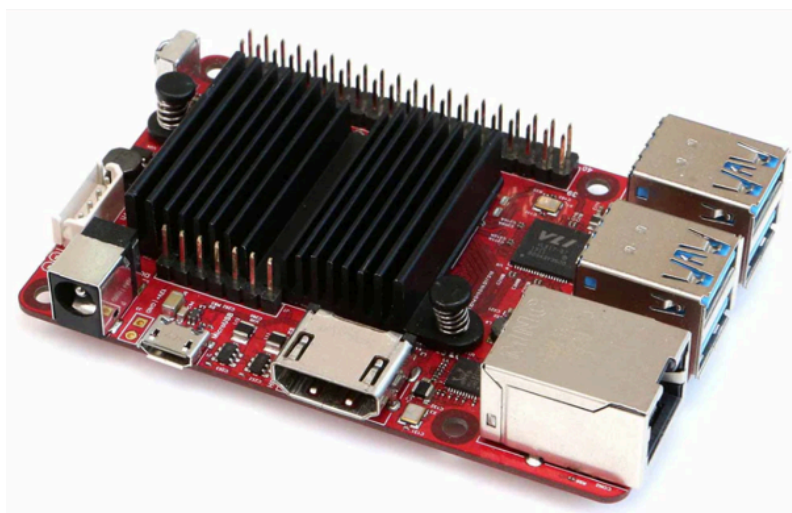
Seria Raspberry Pi jest najszerzej wspieraną platformą dla Home Assistant. Jej popularność wynika z przystępnej ceny i ogromnego wsparcia społeczności. System „Home Assistant OS” (HAOS) jest specjalnie zoptymalizowany dla Raspberry Pi, oferując użytkownikom doświadczenie typu „po prostu działa”.



Rysunek 3: Raspberry Pi pozostaje standardem wejściowym dla entuzjastów automatyki domowej.

2.2.2.2. ODROID-C4

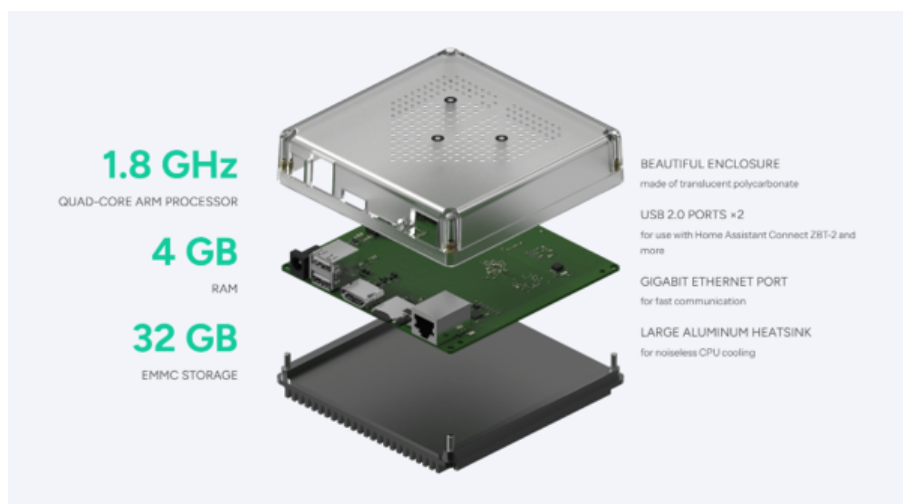
Dla użytkowników wymagających większej stabilności i lepszej wydajności pamięci masowej niż w przypadku konstrukcji Raspberry Pi opartej na kartach microSD, ODROID-C4 stanowi przekonującą alternatywę. Dzięki obsłudze modułów eMMC, ODROID-C4 zapewnia znacznie wyższą niezawodność operacji wejścia/wyjścia dysku, co jest kluczowe dla baz danych SQLite lub MariaDB, których Home Assistant używa do przechowywania historycznych danych o stanie urządzeń.



Rysunek 4: ODROID-C4 stanowi solidną alternatywę z lepszymi opcjami pamięci masowej, takimi jak eMMC.

2.2.3. Dedykowany sprzęt: Home Assistant Green

Dostrzegając potrzebę prostego punktu wejścia, Nabu Casa (firma stojąca za Home Assistant) wprowadziła Home Assistant Green. Urządzenie to zostało zaprojektowane jako najłatwiejszy sposób na rozpoczęcie przygody z Home Assistant, dostarczane z preinstalowanym oprogramowaniem i zoptymalizowane pod kątem wydajności od razu po wyjęciu z pudełka, bez konieczności montażu technicznego.



Rysunek 5: Widok architektoniczny 3D wewnętrznych komponentów sprzętowych Home Assistant Green, prezentujący jego zoptymalizowaną konstrukcję.

2.2.4. Rozwiązania wysokowydajne i klasy enterprise

W miarę jak systemy automatyki domowej rozrastają się i obejmują dziesiątki kamer, złożone modele uczenia maszynowego do wykrywania obiektów oraz liczne usługi zwirtualizowane, ograniczenia komputerów SBC stają się widoczne. W takich przypadkach użytkownicy zwracają się ku mocniejszemu sprzętowi.

2.2.4.1. Komputery osobiste x86-64

Wykorzystanie standardowego komputera PC x86-64 lub dedykowanego serwera pozwala na praktycznie nieograniczoną rozbudowę. Systemy te obsługują szybkie pamięci masowe NVMe, duże ilości pamięci RAM i potężne procesory zdolne do obsługi intensywnych zadań, takich jak transkodowanie wideo w czasie rzeczywistym i automatyzacja oparta na sztucznej inteligencji.



Rysunek 6: Standardowy komputer PC x86-64 oferuje maksymalną elastyczność i wydajność dla instalacji na dużą skalę.

2.2.4.2. Nowoczesne komputery Mini PC: Apple Mac Mini

Najnowsza generacja komputerów Mini PC, której przykładem jest Apple Mac Mini z czipem M4, reprezentuje szczyt wydajności w stosunku do zużytej energii. Urządzenia te oferują wydajność klasy stacji roboczej w obudowie niewiele większej od niektórych komputerów SBC. Dla użytkowników, którzy chcą uruchamiać Home Assistant obok innych ciężkich obciążeń (takich jak serwer mediów czy prywatna chmura), te nowoczesne jednostki zapewniają doskonałą równowagę między mocą a wydajnością.



Rysunek 7: Apple Mac Mini M4 służy jako wysokiej klasy, energooszczędna platforma dla wymagających centrów automatyki domowej.

Krajobraz sprzętowy dla Home Assistant jest zróżnicowany, od przystępnych cenowo Raspberry Pi i Home Assistant Green po wysokowydajne serwery x86-64 i nowoczesne komputery Mini PC. Idealny wybór jest określany przez specyficzne wymagania ekosystemu inteligentnego domu, balansując między kosztem, łatwością obsługi a zapotrzebowaniem na moc obliczeniową.

2.3. Protokół MQTT

Protokół MQTT (ang. **Message Queuing Telemetry Transport**) jest jednym z najpopularniejszych standardów komunikacyjnych wykorzystywanych w świecie Internetu Rzeczy (IoT). Został opracowany w 1999 roku przez Andy'ego Stanforda-Clarka (IBM) oraz Arlena Nipperera (Arcom) z myślą o monitorowaniu rurociągów naftowych za pośrednictwem łącz satelitarnych. Głównym założeniem projektowym była minimalizacja narzutu danych przy jednoczesnym zapewnieniu niezawodności w sieciach o niskiej przepustowości i dużej latencji.

Obecnie MQTT jest standardem otwartym, zarządzanym przez organizację OASIS. Najpowszechniej stosowaną wersją jest 3.1.1, jednak coraz większą popularność zyskuje wersja 5.0, wprowadzająca m.in. ulepszone raportowanie błędów oraz metadane wiadomości (ang. **User Properties**). Lekkość i wydajność protokołu sprawiły, że stał się on de facto standardem w systemach automatyki domowej, w tym w szczególności w ekosystemie Home Assistant.

2.3.1. Architektura i zasada działania

W przeciwieństwie do tradycyjnego modelu klient-serwer (opartego na żądaniach HTTP), MQTT wykorzystuje model **publikacja-subskrypcja** (ang. **publish-subscribe**). W architekturze tej urządzenia końcowe nie komunikują się bezpośrednio ze sobą, lecz za pośrednictwem centralnego węzła zwanego **brokerem**.

Kluczowe komponenty architektury MQTT to:

- **Broker:** Centralny serwer odpowiedzialny za odbieranie wszystkich wiadomości, ich filtrowanie oraz dystrybucję do odpowiednich odbiorców. W kontekście Home Assistant najczęściej wykorzystywanym brokerem jest **Eclipse Mosquitto**.
- **Klient:** Dowolne urządzenie (np. czujnik temperatury, inteligentne gniazdko, smartfon) lub aplikacja, która nawiązuje połączenie z brokerem. Klient może pełnić rolę publikującego, subskrybującego lub obie te role jednocześnie.
- **Temat (Topic):** Ciąg znaków o strukturze hierarchicznej (oddzielanej ukośnikami, np. dom/salon/temperatura), który służy jako adres logiczny wiadomości. Broker wykorzystuje tematy do decydowania, do których klientów ma trafić dana informacja.

Zasada działania polega na tym, że klient subskrybujący informuje brokera o chęci otrzymywania wiadomości z określonych tematów. Gdy inny klient (publikujący) wyśle wiadomość do danego tematu, broker natychmiast przekazuje ją do wszystkich klientów, którzy go subskrybują.

2.3.2. Mechanizmy zapewnienia jakości usług (QoS)

Jedną z najistotniejszych cech MQTT jest możliwość definiowania poziomu jakości usług (ang. **Quality of Service** – QoS), co pozwala na dostosowanie niezawodności dostarczania wiadomości do wymagań konkretnego zastosowania:

- **QoS 0 (At most once):** Wiadomość jest wysyłana tylko raz, bez potwierdzenia odbioru. Jest to najszybszy tryb, ale nie gwarantuje dostarczenia danych (np. dla cyklicznych odczytów temperatury, gdzie utrata jednego pomiaru nie jest krytyczna).
- **QoS 1 (At least once):** Gwarantuje, że wiadomość dotrze do odbiorcy przynajmniej raz. Wymaga potwierdzenia odbioru, a w przypadku jego braku następuje retransmisja.
- **QoS 2 (Exactly once):** Najbardziej zaawansowany i kosztowny pod względem narzutu sieciowego tryb, gwarantujący, że wiadomość dotrze do odbiorcy dokładnie jeden raz, bez duplikacji.

2.3.3. Kluczowe funkcje protokołu

Protokół MQTT oferuje szereg mechanizmów optymalizujących komunikację w systemach inteligentnego domu:

- **Retained Messages (Wiadomości utrwalone):** Broker przechowuje ostatnią poprawną wiadomość wysłaną na dany temat. Dzięki temu nowy klient, tuż po subskrypcji tematu, natychmiast otrzymuje aktualny stan urządzenia, nie czekając na kolejną publikację.
- **Last Will and Testament (LWT):** Pozwala klientowi zdefiniować wiadomość, która zostanie automatycznie opublikowana przez brokera w przypadku nieoczekiwanego rozłączenia klienta (np. utraty zasilania lub zasięgu Wi-Fi). W Home Assistant mechanizm ten jest kluczowy dla oznaczania urządzeń jako niedostępne (**unavailable**).
- **Keep Alive:** Mechanizm cyklicznego sprawdzania połączenia (ping), który pozwala brokerowi i klientowi na szybkie wykrycie zerwania sesji.

2.3.4. MQTT w systemie Home Assistant

W systemie Home Assistant protokół MQTT pełni rolę uniwersalnej magistrali komunikacyjnej, umożliwiającej integrację urządzeń od różnych producentów oraz własnych rozwiązań opartych na mikrokontrolerach (np. ESP8266, ESP32).

Szczególne znaczenie ma mechanizm **MQTT Discovery**. Pozwala on urządzeniom na automatyczne przedstawienie się systemowi Home Assistant poprzez wysłanie specjalnie sformatowanej wiadomości konfiguracyjnej w formacie JSON na określony temat (domyślnie `homeassistant/`). Dzięki temu użytkownik nie musi ręcznie edytować plików konfiguracyjnych – nowe urządzenie pojawia się w interfejsie automatycznie jako gotowa do użycia encja.

Najpopularniejszym sposobem implementacji MQTT w Home Assistant jest instalacja dodatku **Mosquitto broker**, który jest w pełni zintegrowany z systemem.

2.3.5. Bezpieczeństwo w MQTT

Ze względu na to, że MQTT często przesyła dane w sieciach lokalnych i rozległych, kluczowe jest zapewnienie odpowiedniego poziomu bezpieczeństwa. Protokół wspiera:

- **Uwierzytelnianie:** Wymaganie nazwy użytkownika i hasła przy nawiązywaniu połączenia z brokerem.
- **Autoryzację (ACL):** Definiowanie uprawnień dla poszczególnych użytkowników (np. dany klient może tylko subskrybować konkretny temat, ale nie może na nim publikować).
- **Szyfrowanie TLS/SSL:** Zabezpieczenie całej komunikacji przed podsłuchem i atakami typu **Man-in-the-Middle**.

Wykorzystanie MQTT w połączeniu z oprogramowaniem takim jak **Tasmota**, **Zigbee2MQTT** czy **ESPHome** stanowi fundament zaawansowanych i stabilnych instalacji inteligentnego domu.

2.4. Docker

Docker to otwarta platforma służąca do automatyzacji wdrażania, skalowania i zarządzania aplikacjami wewnątrz lekkich i przenośnych kontenerów. Została ona zapoczątkowana przez Solomona Hykesa i udostępniona jako projekt open-source w marcu 2013 roku. Docker zrewolucjonizował podejście do tworzenia oprogramowania, wprowadzając standard konteneryzacji, który pozwala na izolację aplikacji od infrastruktury systemowej, zapewniając ich spójne działanie w różnych środowiskach – od lokalnej maszyny programisty po zaawansowane klastry obliczeniowe w chmurze.

2.4.1. Konteneryzacja a Wirtualizacja

Kluczem do zrozumienia fenomenu Dockera jest rozróżnienie między tradycyjną wirtualizacją a konteneryzacją. Tradycyjne maszyny wirtualne (VM) wymagają pełnego systemu operacyjnego (Guest OS) dla każdej instancji, co wiąże się z dużym narzutem zasobów (procesora, pamięci RAM i miejsca na dysku).

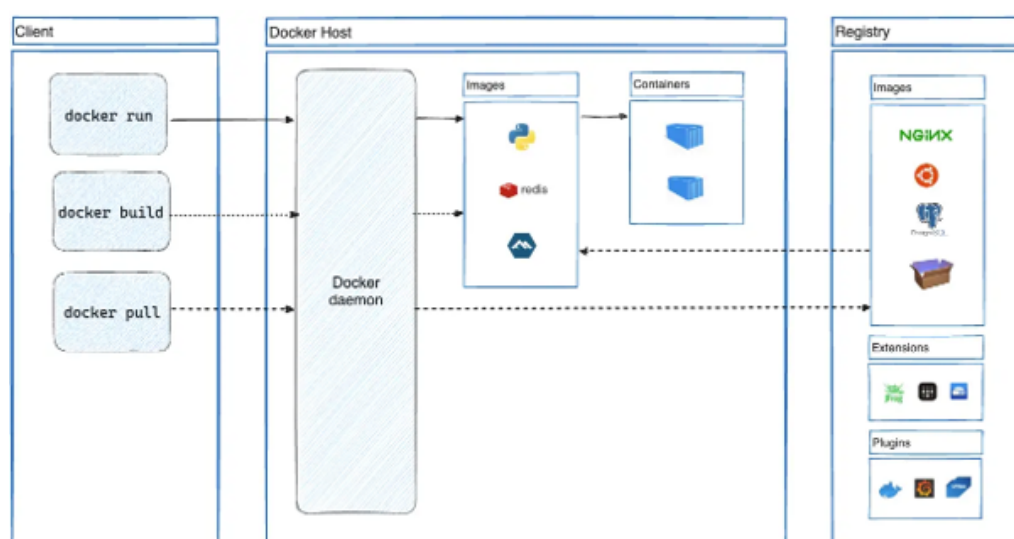
Konteneryzacja, na której opiera się Docker, wykorzystuje mechanizmy jądra systemu operacyjnego gospodarza (takie jak namespaces i control groups w systemie Linux) do izolacji procesów. Dzięki temu kontenery współdzielą jądro systemu z gospodarzem,

co czyni je znacznie lżejszymi, szybszymi w uruchamianiu i bardziej wydajnymi pod względem wykorzystania zasobów niż maszyny wirtualne.

2.4.2. Architektura Systemu Docker

Architektura Dockera opiera się na modelu klient-serwer (rysunek Rysunek 8). Głównymi elementami tego systemu są:

- **Docker Daemon (dockerd):** Działa w tle na systemie gospodarza i zarządza obiektami Dockera, takimi jak obrazy, kontenery, sieci oraz wolumeny.
- **Docker Client:** Narzędzie linii komend (CLI), które pozwala użytkownikowi komunikować się z demonem Dockera poprzez API REST.
- **Docker Registry:** Miejsce przechowywania obrazów. Najpopularniejszym publicznym rejestrem jest Docker Hub, ale organizacje często korzystają z prywatnych rejestrów w celu zapewnienia bezpieczeństwa.



Rysunek 8: Architektura platformy Docker i interakcja między jej komponentami

2.4.3. Kluczowe Pojęcia i Obiekty

W ekosystemie Dockera wyróżnia się kilka fundamentalnych pojęć, które stanowią o jego funkcjonalności:

- **Obraz (Image):** Tylko do odczytu szablon instrukcji służący do tworzenia kontenera. Często obraz bazuje na innym obrazie z dodatkowymi modyfikacjami (np. obraz aplikacji bazujący na obrazie systemu Ubuntu z zainstalowanym serwerem Apache).
- **Kontener (Container):** Uruchomiona instancja obrazu. Można go tworzyć, uruchamiać, zatrzymywać, przenosić lub usuwać. Kontenery są z definicji ulotne, co oznacza, że dane w nich zapisane zostają utracone po ich usunięciu, chyba że zostaną użyte mechanizmy trwałości danych (wolumeny).
- **Dockerfile:** Plik tekstowy zawierający zestaw instrukcji, które Docker wykonuje w celu automatycznego zbudowania obrazu. Pozwala to na pełną dokumentację środowiska i łatwą reprodukcję artefaktów.
- **Wolumeny (Volumes):** Mechanizm służący do trwałego przechowywania danych generowanych i używanych przez kontenery. Wolumeny są zarządzane przez Dockera i są niezależne od cyklu życia samego kontenera.

2.4.4. Zalety Wykorzystania Dockera w Projektach Inżynierskich

Zastosowanie Dockera w procesie wytwórczym oprogramowania niesie ze sobą szereg korzyści:

1. **Izolacja środowiska:** Każdy kontener posiada własne biblioteki i zależności, co eliminuje problem „u mnie działa” oraz konflikty między różnymi wersjami oprogramowania na tej samej maszynie.
2. **Przenośność:** Raz zbudowany obraz może zostać uruchomiony na dowolnym systemie wspierającym Dockera, bez konieczności dodatkowej konfiguracji.
3. **Szybkość i wydajność:** Kontenery uruchamiają się w ciągu milisekund, co pozwala na błyskawiczne skalowanie usług i optymalne wykorzystanie infrastruktury.
4. **Usprawnienie CI/CD:** Docker idealnie wpisuje się w potoki ciągłej integracji i ciągłego wdrażania, umożliwiając automatyczne budowanie i testowanie identycznych artefaktów na każdym etapie cyklu życia aplikacji.

Dzięki tym cechom Docker stał się standardem de facto w nowoczesnym inżynierii oprogramowania, stanowiąc fundament dla architektury mikroserwisowej oraz nowoczesnych systemów automatyki i IoT.

3. Praca własna

3.1. Przygotowanie środowiska serwerowego

3.1.1. Instalacja systemu operacyjnego Debian na laptopie Lenovo T470

Pierwszym etapem prac praktycznych było przygotowanie środowiska systemowego. Zdecydowano się na wybór dystrybucji Debian GNU/Linux w wersji 12 (Bookworm), która charakteryzuje się wysoką stabilnością oraz szerokim wsparciem społeczności, co jest kluczowe w projektach o charakterze badawczym i inżynierskim.

3.1.1.1. Charakterystyka sprzętowa

Proces instalacji przeprowadzono na laptopie Lenovo ThinkPad T470. Model ten jest powszechnie ceniony za wysoką kompatybilność z systemami z rodziny Linux. Kluczowe parametry techniczne wykorzystanego urządzenia zestawiono poniżej:

- Procesor: Intel Core i5-7300U,
- Pamięć RAM: 16 GB DDR4,
- Dysk: 256 GB SSD NVMe,
- Karta sieciowa: Intel Dual Band Wireless-AC 8265.



Rysunek 9: Widok ogólny laptopa Lenovo ThinkPad T470 wykorzystanego w badaniach.

3.1.1.2. Przygotowanie nośnika instalacyjnego

Proces rozpoczęto od pobrania oficjalnego obrazu ISO typu „netinst” z serwerów projektu Debian. Wykorzystanie obrazu sieciowego umożliwiło instalację najnowszych wersji pakietów już na etapie wdrażania systemu. Obraz został przeniesiony na nośnik USB o pojemności 16 GB przy użyciu systemowego narzędzia dd w środowisku Linux. Operację wykonano przy użyciu następującego polecenia:

```
sudo dd if=debian-12-netinst-amd64.iso of=/dev/sdX bs=4M status=progress  
oflag=sync
```

3.1.1.3. Konfiguracja BIOS/UEFI

Przed przystąpieniem do właściwej instalacji konieczne było dostosowanie ustawień oprogramowania układowego (UEFI) urządzenia. Kluczowe modyfikacje objęły:

1. Wyłączenie funkcji **Secure Boot**, co zapewnia bezproblemowe ładowanie modułów jądra.
2. Ustawienie trybu pracy kontrolera pamięci masowej na **AHCI**.
3. Zmianę priorytetu urządzeń rozruchowych, ustawiając nośnik USB jako nadrzędny.

3.1.1.4. Przebieg instalacji

Po uruchomieniu komputera z przygotowanego nośnika wybrano tryb **Graphical Install**. Proces przebiegał zgodnie z następującymi etapami:

1. Konfiguracja regionalna: Wybrano język polski oraz układ klawiatury programisty.
2. Konfiguracja sieci: Dzięki zintegrowaniu w jądrze systemu Debian 12 sterowników z sekcji **non-free-firmware**, karta sieciowa Intel została poprawnie zidentyfikowana, co umożliwiło nawiązanie połączenia bezprzewodowego w trakcie instalacji.
3. Partycjonowanie dysku: Zastosowano automatyczne partycjonowanie całej przestrzeni dyskowej z wykorzystaniem mechanizmu LVM (Logical Volume Manager). Wybór ten podyktowany był elastycznością w przyszłym zarządzaniu wolumenami logicznymi.
4. Wybór oprogramowania: Zdecydowano o pominięciu instalacji środowiska graficznego. Wybrano jedynie podstawowe narzędzia systemowe (**Standard system utilities**) oraz serwer SSH (**SSH server**). Takie podejście pozwoliło na minimalizację narzutu systemowego i zwiększenie poziomu bezpieczeństwa stacji roboczej.

3.1.1.5. Konfiguracja poinstalacyjna

Po pomyślnym zakończeniu procesu i ponownym uruchomieniu urządzenia, przeprowadzono kroki konfiguracyjne:

- Dodanie konta użytkownika do grupy `sudo`, umożliwiając zarządzanie systemem z uprawnieniami administratora.
- Weryfikacja pliku `/etc/apt/sources.list` pod kątem obecności repozytoriów `main`, `contrib` oraz `non-free-firmware`.
- Wykonanie pełnej aktualizacji pakietów systemowych poleceniem `sudo apt update && sudo apt upgrade`.

System Debian 12 na platformie Lenovo T470 wykazał pełną kompatybilność sprzętową bezpośrednio po instalacji, co stanowi stabilny fundament dla dalszych prac badawczych.

3.1.2. Konteneryzacja środowiska przy użyciu platformy Docker

Po skonfigurowaniu systemu operacyjnego Debian, kolejnym krokiem było wdrożenie platformy Docker. Konteneryzacja pozwala na izolację poszczególnych usług (takich jak broker MQTT czy Home Assistant), co znacząco ułatwia zarządzanie zależnościami oraz migrację systemu między różnymi platformami sprzętowymi.

3.1.2.1. Instalacja silnika Docker Engine

Proces instalacji przeprowadzono zgodnie z oficjalną dokumentacją, wykorzystując repozytoria producenta w celu zapewnienia dostępu do najnowszych wersji oprogramowania.

1. **Aktualizacja indeksu pakietów i instalacja zależności:** W pierwszej kolejności zainstalowano pakiety umożliwiające bezpieczną komunikację z repozytoriami przez protokół HTTPS:

```
sudo apt update
sudo apt install ca-certificates curl gnupg
```

2. **Dodanie oficjalnego klucza GPG Docker:** Klucz służy do weryfikacji autentyczności pobieranych pakietów:

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --
dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

3. **Konfiguracja repozytorium:** Dodano wpis do listy źródeł systemowego menedżera pakietów apt:

```
echo \
  "deb [arch="$(dpkg --print-architecture)" signed-by=/etc/apt/
keyrings/docker.gpg] https://download.docker.com/linux/debian \
  "$(. /etc/os-release && echo "$VERSION_CODENAME")" stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

4. **Instalacja pakietów Docker:** Po zaktualizowaniu listy repozytoriów, zainstalowano silnik Docker oraz wtyczkę Docker Compose:

```
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-
plugin docker-compose-plugin
```

3.1.2.2. Konfiguracja poinstalacyjna

Domyślnie demon Docker wymaga uprawnień administratora. Aby umożliwić zarządzanie kontenerami bez każdorazowego użycia polecenia `sudo`, dodano bieżącego użytkownika do grupy `docker`:

```
sudo usermod -aG docker $USER
```

Zmiana ta wymagała ponownego zalogowania się do systemu, aby nowa przynależność do grupy została uwzględniona w bieżącej sesji.

3.1.2.3. Weryfikacja poprawności instalacji

Poprawność wdrożenia zweryfikowano poprzez uruchomienie testowego kontenera `hello-world`:

```
docker run hello-world
```

Pomyślne wykonanie tego polecenia potwierdziło, że silnik Docker jest poprawnie zainstalowany, a użytkownik posiada niezbędne uprawnienia do zarządzania kontenerami.

3.1.2.4. Wykorzystanie Docker Compose

W dalszych etapach pracy, zamiast pojedynczych poleceń `docker run`, zdecydowano się na wykorzystanie narzędzia Docker Compose. Pozwala ono na definiowanie całej

infrastruktury w formie deklaracyjnych plików `docker-compose.yml`, co zapewnia powtarzalność konfiguracji i ułatwia dokumentowanie wdrożonych usług.

3.2. Uruchomienie brokera MQTT

...

3.3. Postawienie Home Assistant

...

3.4. Analiza ruchu sieciowego

...

4. Podsumowanie

Bibliografia